Module 1 - Quality Assessment and Genome Assembly

A hands-on bioinformatics workshop using Salmonella Typhi sequencing data

Table of contents

Introduction

This module is a hands-on introduction to two foundational stages of any bacterial whole-genome-sequencing analysis: **quality assessment** of raw sequencing reads, and **de novo assembly** of those reads into draft genomes. We will work end-to-end with a set of *Salmonella enterica* serovar Typhi isolates. These sequences were collected as part of the SEAP study in Nepal and they are also available from the European Nucleotide Archive (ENA).

Salmonella Typhi is the causative agent of typhoid fever, an enteric infection that remains a major public health concern in many parts of Asia, Africa, and Latin America. Routine whole-genome sequencing of clinical Typhi isolates supports outbreak investigation, antimicrobial resistance surveillance, and lineage tracking.

1 Module setup

By the end of this module you will have run a complete QC pipeline: FastQC → fastp → minimap2 → BactInspector → shovill → QUAST on real public-health-relevant data, and you will be able to interpret the outputs of each step and decide whether your data is fit for downstream analysis.

1.1 Learning objectives for the module

By the end of this module, you will be able to:

- Inspect the quality of raw Illumina paired-end reads with **FastQC**.
- Trim sequencing adapters and low-quality bases with **fastp**.
- Remove host (human) reads from a bacterial isolate dataset with **minimap2** and **samtools**.
- Confirm species identity with **BactInspector**.
- Assemble draft bacterial genomes from short reads with **shovill**.
- Evaluate assembly quality with **QUAST** using both reference-free and reference-based metrics.
- Aggregate per-sample results across the whole pipeline with **MultiQC** and investigate the resulting summary dashboard.

1.2 The dataset

The dataset we will be working with consists of Illumina paired-end sequencing reads of *Salmonella* Typhi isolates. Each isolate is represented by **two** FASTQ files, one for forward reads and one for reverse reads of the paired-end set. The files should look like this:

```
Typhi_fastq/
  <ACCESSION_1>_1.fastq.gz   # forward reads, sample 1
  <ACCESSION_1>_2.fastq.gz   # reverse reads, sample 1
  <ACCESSION_2>_1.fastq.gz
  <ACCESSION_2>_2.fastq.gz
  ...
  <ACCESSION_X>_2.fastq.gz
```

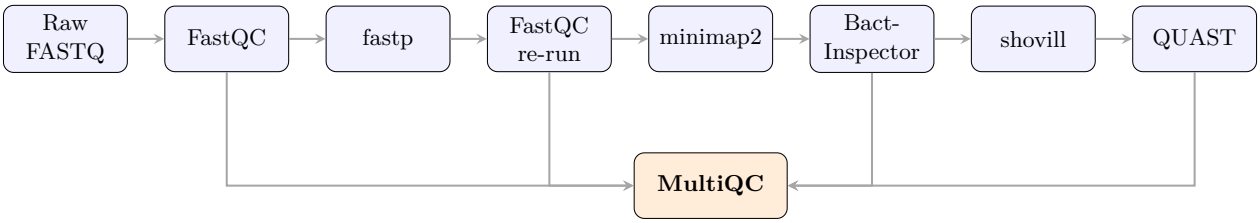
Exercise 1.1 — Inspect the dataset

Open a terminal and navigate to the workshop working directory (`cd` to the workshop directory). Then:

1. List the contents of the `Typhi_fastq/` directory (`ls -l`).
2. Count the number of `.fastq.gz` files present (hint: you can combine `ls` and `wc -l`).
3. Confirm that for every forward reads file (`_1`) there is a matching reverse reads file (`_2`).

1.3 The module at a glance

Over the course of this module, we will run the following workflow, one tool at a time. Each step takes the output of the previous step as its input.



The two **MultiQC** runs give us a "before and after" view across all samples in a single HTML page.

1.4 Software environment

All the tools we will use today are pre-installed in a single **conda environment** called `typhi-qc`. Conda is a package and environment manager: rather than installing tools globally on the laptop, conda keeps each project's tools in a self-contained environment and lets you activate it when you need it. This avoids version conflicts between tools.

Activate the workshop environment now:

CLI

```
conda activate typhi-qc
```

Your shell prompt should change to indicate the environment is active, typically by adding `(typhi-qc)` at the front:

```
(typhi-qc) user@host:~/typhi-workshop$
```

If the prompt does not change and you see *command not found* errors, conda may not be initialised for your shell. Speak to an instructor if this is the case and try the `conda activate` command again.

1.5 Working directory layout

We recommend organising your work directory as follows. The `Typhi_fastq/` and `ref/` directories already exist and contain the data and reference files you need; you will create the `results/` subdirectories as we go - an example is given below.

```
~/typhi-workshop/
   Typhi_fastq/               # raw paired-end FASTQ files (provided)
       <ACCESSION>_1.fastq.gz
       <ACCESSION>_2.fastq.gz
       ... (30 files total)
   ref/                       # reference data (provided)
       GRCh38_subset.mmi      # minimap2 index for host filtering
       CT18_reference.fa      # Salmonella Typhi CT18 reference genome
   results/                   # to be created during the workshop
       fastqc/
       fastp/
       fastqc_trimmed/
       host_filtered/
       bactinspector/
       assemblies/
       quast/
       multiqc/
```

Create the `results` directory:

CLI

```
mkdir results
```

1.6 Verifying the tools

Before we start any analysis, confirm that every tool we will use today is available in the activated environment by checking each one's version string. The exact version numbers do not matter much for this workshop, but the commands should all run without errors.

CLI

```
fastqc --version
fastp --version
minimap2 --version
samtools --version | head -n 1
bactinspector --version
shovill --version
quast.py --version
multiqc --version
```

Each command should print a single version line. It is good practice to make a note of the tool version you used for each of your analysis. If any command returns *command not found*, the environment is incomplete. Speak to an instructor for installing the missing tools.

1.7 Check your understanding

? Check Your Understanding 1.1

a) Why do paired-end Illumina runs produce *two* FASTQ files per sample rather than one?
b) Looking at the pipeline diagram, FastQC appears twice. What is the purpose of the second FastQC run?
c) Why do we use a conda environment rather than installing each tool globally on the laptop?

2 Background — NGS data and quality control fundamentals

2.1 Illumina sequencing overview

A single Illumina sequencing run produces tens of millions of short reads, typically 75–300 bases long, drawn from random positions across the input DNA. Each read carries a per-base quality score that reflects how confident the basecaller was about that base.

For the Typhi isolates we will analyse, the sequencing was done in **paired-end** mode. That means each DNA fragment is sequenced from both ends, producing two reads per fragment: a *forward* read (R1) starting at one end, and a *reverse* read (R2) starting at the other. The reads point inward toward each other and the gap between them (the **insert size**) is typically a few hundred bases. Paired-end reads are far more useful than single-end reads for two reasons: they give us two independent observations of each fragment, and the constraint that the two reads must map to nearby positions on the genome in opposite orientations is a powerful disambiguator during alignment and assembly.

The amount of sequencing data per sample is described by **sequencing depth**, calculated as the total bases sequenced divided by the size of the reference (or expected) genome (i.e., the number of times that a single base in a genome is sequenced). For a *Salmonella* Typhi genome of ~4.8 Mb, a sample with 2 million 150-bp paired-end reads has a depth of roughly $\frac{2{,}000{,}000 \times 150 \times 2}{4{,}800{,}000} \approx 125\times$. Typical Typhi data sit in the 30× to 200× range; below 30× assembly quality suffers, above 100× there are diminishing returns and we often downsample for speed. **Coverage** referes to the presentage of the target genome that is read at least once (or whatever the threshold is set).

2.2 FASTA and FASTQ data formats

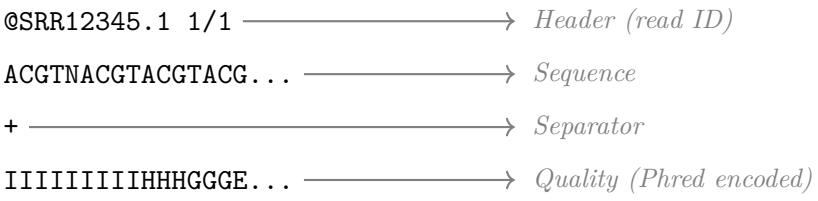
2.2.1 FASTA format

FASTA files are used to store both nucleotide and protein sequences. Each sequence is preceded by its header line which always starts with a > and contains the sequence name and/or a unique identifier. Each FASTA file can contain one or more than one sequences. For example:

```
>Seq1
GTCCGAAATCGTTTAA
>Sequence-2
TTTAAACCCGAATGCCAACCGGTTAACCGG
```

2.2.2 FASTQ format

Every read produced by an Illumina sequencer is stored in a **FASTQ** file. The format is plain text, four lines per read, with a strict structure. It contains quality information for each base.



The first line (**header**) starts with `@` and contains a unique identifier for the read, often encoding the instrument, flowcell lane, tile, and coordinates; it is often terminated with a /1 or /2 to show that the read is part of a pair. The second line is the **sequence** itself, written using the IUPAC nucleotide alphabet (`A`, `C`, `G`, `T`, and `N` for any base, called ambiguously). The third line is just a `+` separator, occasionally followed by a repeat of the header. The fourth line is the **quality string** containing the per base Phred quality score (one ASCII character per base in the sequence, in the same order).

Because FASTQ files are large, they are almost always gzip-compressed with a `.fastq.gz` extension. Most bioinformatics tools read gzipped FASTQ transparently, so you rarely need to decompress them manually. If you do want to peek, use `zcat` or `zless`:

CLI

```
zcat Typhi_fastq/ERR4025225_1.fastq.gz | head -n 4
```

It should look something like this:

```
@ERR4025225.1`HX4_28933:2:1101:22495:1186/1
NGCGAAGCGCTGGCCGTGGATGCGAACGGCCAGCTCCTGTCCCGATATGAGGCAGGAATATGGAAAATCATTCCCCCGTCGG
ATTTTGCCCGCGACGTGGCCGGGTTGTTTCAGCGTCTGCGCGCCCCGTTCTCGTCGGGGAAAATTGCCT
+
!AAFFJJJJJJJJJJJJJJJJJJJJJJJJJJJJFJJJJJJJJJJJJJJJJJJAJJ<FJJJJJJFJJFJJJJJJJJJJJJJ
FJJJJJJJJJJFJJJJJJJJJJFJJFJJJJJJJJJ<JJJJJFFJJJAFAFFFFF<7JFA<FFFJJFJ7
```

2.3 Phred quality scores

The quality string tells us a lot about this FASTQ record. Each character encodes a **Phred quality score**, Q, defined as

$$Q = -10 \log_{10}(P)$$

where P is the probability that the base call at that position is wrong. A Q score of 30 means the basecaller estimates a 1:1000 chance that the base is incorrect, i.e., 99.9% accuracy. Higher Q is better.

Q value	Error rate	Accuracy	ASCII char
Q10	1 in 10	90%	`+`
Q20	1 in 100	99%	`5`
Q30	1 in 1,000	99.9%	`?`
Q40	1 in 10,000	99.99%	`I`

To decode a quality character to its Q score: take the ASCII value of the character and subtract 33. So `I` (ASCII 73) decodes to Q40, `?` (ASCII 63) decodes to Q30, `5` (ASCII 53) decodes to Q20. Most modern Illumina runs produce reads where the bulk of the bases are Q30 or higher, with a gradual decline in quality toward the 3' end of each read.

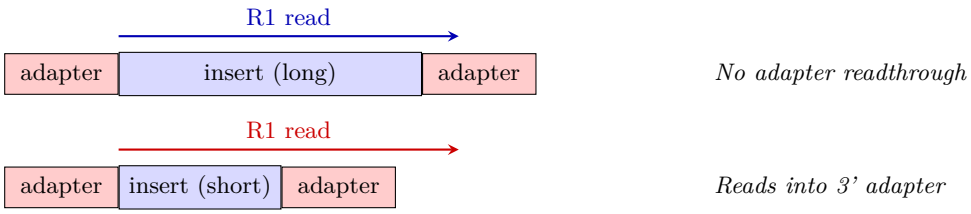
> Exercise 2.1 — Decode a quality string
>
> The quality string for the first 10 bases of a read is:
>
> `JJJIIH?<.5`
>
> For each character, work out the Q score (ASCII value minus 33) and the corresponding error rate. You can use `man ascii` in the terminal to look up ASCII values, or just remember a few anchor points: `!` is ASCII 33 (Q0), `5` is 53 (Q20), `?` is 63 (Q30), `I` is 73 (Q40), `S` is 83 (Q50). Comment on what the trend says about basecall confidence across the first 10 bases.

2.4 Common quality-control issues

Almost every short-read sequencing run has some imperfections. Identifying them is the first job of QC.

Adapter readthrough. Illumina reads start at a known oligonucleotide adapter and proceed into the actual DNA insert. If the insert is shorter than the read length, the sequencer reads past the end of the insert and into the adapter on the other side, contaminating the 3' end of the read with

adapter sequence. This is especially common when fragment libraries are smaller than expected. Trimming removes this contamination.



Quality drop at the 3' end. Illumina chemistry loses fidelity over the length of a read. The first 30–50 bases are usually very high quality (Q35+); the last 20–30 bases of a 150-bp read are often Q20 or lower. We typically trim the low-quality 3' tail.

Per-base sequence content bias. Per-base sequence content bias is an uneven distribution of A, C, T, and G nucleotides across read positions. While a random library should show roughly parallel proportions of all four bases, technical artifacts during library preparation (like random hexamer priming during Illumina library prep) often cause imbalances, particularly at the 5' ends of reads. This can show up as a "fail" in FastQC's "Per base sequence content" module, and it is not a problem for downstream analysis.

GC content shifts. A bacterial genome has a characteristic GC content (Typhi is ~52%). If the GC distribution across reads deviates strongly from the expected, it can indicate contamination with another organism or a fragment-bias artefact in library prep.

Sequence duplication. Some level of duplication is normal; for high-coverage runs you'll have multiple reads from nearby positions on the genome by chance. However, problematic high duplication (>40%) often points to PCR over-amplification during library prep.

Overrepresented sequences and contamination. Specific sequences appearing in >1% of reads is unusual for a random-sheared library and often points to adapter contamination, primer dimers, or host DNA.

2.5 Check your understanding

? Check Your Understanding 2.1

a) A FASTQ record has a sequence line of length 150. How long must the quality line be? What does it mean if they differ?
b) You see a read where the first 40 quality characters are `I` and the last 110 are progressively lower (down to `5` at the end). Roughly what does this say about the quality profile of the read? Should you trim, and from which end?
c) Adapter readthrough produces contamination at the 3' end of reads, not the 5' end. Why is that, given that the adapter is also at the 5' end of every fragment?

3 Initial quality control with FastQC

We will run **FastQC** on all samples, read its HTML report, and then aggregate the individual reports into one summary view with **MultiQC**. By the end of the chapter you will be able to look at a FastQC report and say something concrete about whether the data is good, bad, or somewhere in between.

3.1 Why run FastQC?

FastQC is a reporting tool, it scans a FASTQ file and produces a per-sample HTML report covering several quality dimensions, but it does not change the data in any way. Running FastQC before any trimming or filtering is essential because it tells you *what* problems your data has, which in turn tells you what trimming parameters to use and what to expect downstream. Running it again *after* trimming lets you confirm that the trimming did what you wanted.

The FastQC report has several sections, each addressing one aspect of read quality. The most useful ones are:

- **Basic Statistics**: total read count, read length, GC content. A first sanity check.
- **Per base sequence quality**: boxplot of Q scores at each position along the read.
- **Per sequence quality scores**: distribution of mean per-read Q score. Tells you whether bad reads are an isolated minority or a widespread problem.
- **Per base sequence content**: proportion of A/C/G/T at each position. The first 10–13 bases may look biased; the rest should be roughly flat.
- **Per sequence GC content**: distribution of per-read GC content vs the expected unimodal distribution. Bimodality often indicates contamination.
- **Sequence Duplication Levels**: proportion of reads observed multiple times.
- **Overrepresented sequences**: specific sequences appearing in >1% of reads.
- **Adapter Content**: fraction of each known adapter at each position along the read.

Each metric is assigned a status of pass (green tick), warn (yellow exclamation), or fail (red cross) based on built-in thresholds. These thresholds are deliberately strict and were designed for a generic Illumina experiment, but with experience some of the warning or even fail messages may be acceptable. Interpret these statuses as guides, not verdicts.

3.2 Running FastQC

We will start with one sample to see the output structure before scaling up. Replace `<ACCESSION>` with the accession of the first sample in your `Typhi_fastq/` directory.

CLI

```
fastqc \
  Typhi_fastq/ERR4025225_1.fastq.gz \
  Typhi_fastq/ERR4025225_2.fastq.gz \
  --outdir results/fastqc/ \
  --threads 2
```

You should see progress messages for each file.

Each input file produces two outputs:

CLI

```
ls results/fastqc/
```

```
ERR4025225_1_fastqc.html
ERR4025225_1_fastqc.zip
ERR4025225_2_fastqc.html
ERR4025225_2_fastqc.zip
```

The HTML is the human-readable report; the ZIP contains the raw data behind the plots, which is what MultiQC reads when it aggregates reports across samples. Open one of the HTML files in your browser (or by double-clicking from your File browser):

CLI

```
xdg-open results/fastqc/ERR4025225_1_fastqc.html
```

Take a few minutes to scroll through every section.

3.3 Running FastQC on all samples

Once you understand the output for one sample, scaling up is straightforward. FastQC accepts multiple input files and parallelises across them with the `--threads` flag:

CLI

```
fastqc Typhi_fastq/*.fastq.gz \
  --outdir results/fastqc/ \
  --threads 4
```

This will take a few minutes to run, depending on disk speed and CPU. While it runs, this is a good moment to read the FastQC manual descriptions for each section, which can be found here https://www.bioinformatics.babraham.ac.uk/projects/fastqc/Help/3%20Analysis%20Modules/.

When it finishes, you will have an HTML file for each fastq file in `results/fastqc/`. Opening so many reports one at a time is impractical, which is where MultiQC comes in.

3.4 Aggregating with MultiQC

MultiQC scans a directory for outputs from supported bioinformatics tools, including FastQC, and produces a single HTML report summarising all samples on each metric.

CLI

```
multiqc results/fastqc/ \
  --outdir results/multiqc/ \
  --filename multiqc_report_rawreads
```

The output is `results/multiqc/multiqc_report_rawreads.html`. Open it in your browser. You will see various summary plots that combine all files that we ran. Hovering over a point in any plot tells you which sample it corresponds to. We will run MultiQC again at later stages after processing the files.

3.5 Exercises

Exercise 3.1 — Generate FastQC reports for all samples

Run FastQC on all raw FASTQ files in `Typhi_fastq/`, then run MultiQC on the resulting output directory. Confirm that:

1. `results/fastqc/` contains the expected number of HTML files.
2. `results/multiqc/multiqc_report_rawreadst.html` opens in your browser and shows all samples.
3. Using the MultiQC report, inspect the Phred scores plot. How does it look for all samples? Do you trust all sequences?
4. How does the duplication level look across samples? Comment on whether you think this duplication is problematic.
5. Is there notable adapter contamination?
6. Decide on whether there is any sequence you would remove from downstream analysis.

3.6 Check your understanding

? Check Your Understanding 3.1

a) What is the difference between a warn and a fail status in FastQC? Are these thresholds absolute, or relative to your data?
b) The "Per base sequence content" module fails on every one of your samples, but only because of the first 10 bases — the rest of the read is flat. Should you be concerned? Why or why not?
c) A colleague is sequencing PCR amplicons (not whole-genome data) and shows you a FastQC report with 80% sequence duplication. Should they panic? Explain.

3.7 Further reading

There are alternative tools worth knowing about, even though we will not use them here:

- **Trim Galore**: a wrapper around Cutadapt and FastQC that combines trimming and QC in one pass. Convenient for simple workflows but less flexible than fastp + FastQC separately.
- **NanoPlot / NanoFilt**: for Oxford Nanopore long reads (different quality profile, different tools).
- **seqkit stats**: for quick numerical summaries (read count, length distribution, GC) without the full FastQC plotting overhead.

4 Adapter and quality trimming with fastp

Now that we have characterised the raw data, we will clean it up. Trimming has two goals: remove any adapter sequence that has read into the 3' end of the reads, and remove low-quality 3' tails where the basecaller has lost confidence. We will use **fastp**, an all-in-one tool that performs adapter detection, quality trimming, and read filtering in a single pass and produces both an HTML and a JSON report.

4.1 Why trim, and what we are trimming

Some issues that can benefit from trimming are:

- **Adapter readthrough.** As previously mentioned, when the insert is shorter than the read length the sequencer continues into the opposite adapter. The contamination is concentrated at the 3' end of the read. If left untrimmed, downstream alignment tools may try to align the adapter sequence to the reference, producing spurious mismatches; assemblers can be confused by the apparent shared sequence between the ends of different fragments.
- **Low-quality 3' tails.** Illumina basecall quality drops over the read length. Bases below a quality threshold are more likely to be wrong, and including them in alignment or assembly introduces noise. We typically trim from the 3' end until quality recovers.
- **polyG tails (NovaSeq/NextSeq).** Two-channel chemistry instruments interpret "no signal" as a G base, so when a cluster fails late in a run you get a stretch of false-positive G's at the end of the read. Specialised polyG detection is part of fastp's default behaviour.

A common worry for beginners is that trimming will remove "real" data. The risk can be real but small if you use sensible parameters. The cost of over-trimming is some loss of yield (a few percent of bases); the cost of under-trimming can be misassembly or false variant calls. A conventional balance is to trim moderately and verify with a re-run of FastQC.

4.2 fastp

fastp's defaults are reasonable for most Illumina paired-end work:

- It auto-detects the adapter sequence using paired-end overlap analysis, which works even when the adapter is unknown or non-standard.
- It quality-trims the 3' end of each read by sliding-window Q-score thresholding.
- It filters out reads that become too short after trimming, or whose mean quality drops below a threshold.
- It produces an HTML and a JSON report; the JSON is what MultiQC picks up.

A typical command for paired-end Illumina is:

```
fastp --in1 Typhi_fastq/<ACCESSION>_1.fastq.gz \
--in2 Typhi_fastq/<ACCESSION>_2.fastq.gz \
--out1 results/fastp/<ACCESSION>_1.trim.fastq.gz \
--out2 results/fastp/<ACCESSION>_2.trim.fastq.gz --detect_adapter_for_pe \
--qualified_quality_phred 20 --length_required 50 \
--html results/fastp/<ACCESSION>_fastp.html \
--json results/fastp/<ACCESSION>_fastp.json --thread 4
```

What these options mean:

- `--detect_adapter_for_pe` enables overlap-analysis adapter detection (more sensitive than the default known-adapter matching).
- `--qualified_quality_phred 20` sets the per-base quality threshold to Q20.
- `--length_required 50` discards any read shorter than 50 bp after trimming.
- `--html` / `--json` write the per-sample QC reports.

Now run for one sample:

CLI

```
fastp \
  --in1  Typhi_fastq/ERR4025225_1.fastq.gz \
  --in2  Typhi_fastq/ERR4025225_2.fastq.gz \
  --out1 results/fastp/ERR4025225_1.trim.fastq.gz \
  --out2 results/fastp/ERR4025225_2.trim.fastq.gz \
  --detect_adapter_for_pe --qualified_quality_phred 20
  --length_required 50 \
  --html results/fastp/ERR4025225_fastp.html \
  --json results/fastp/ERR4025225_fastp.json \
  --thread 4
```

You will see the progress while it is running, as well as key metrics before and after filtering along with the filtering result. Skim those numbers: how many reads survived, how many bases were trimmed off, how the Q20 and Q30 percentages improved.

4.3 Looping over all samples

A `for` loop is the most straightforward way to run fastp on every sample. The loop below assumes the accession is everything before `_1.fastq.gz` in the filename.

CLI

```
for R1 in Typhi_fastq/*_1.fastq.gz; do
  acc=$(basename "$R1" _1.fastq.gz)
  fastp \
    --in1  Typhi_fastq/${acc}_1.fastq.gz \
    --in2  Typhi_fastq/${acc}_2.fastq.gz \
    --out1 results/fastp/${acc}_1.trim.fastq.gz \
    --out2 results/fastp/${acc}_2.trim.fastq.gz \
    --detect_adapter_for_pe --qualified_quality_phred 20 \
    --length_required 50 \
    --html results/fastp/${acc}_fastp.html \
    --json results/fastp/${acc}_fastp.json \
    --thread 4
done
```

This will take a few minutes to run. When it finishes, `results/fastp/` should contain output files that include trimmed FASTQs (paired) and reports (HTML + JSON).

4.4 Re-running FastQC on the trimmed reads

The most direct way to confirm the trimming worked is to run FastQC again on the trimmed output and compare side-by-side with the raw reports.

CLI

```
mkdir results/fastqc_trimmed
fastqc results/fastp/*.trim.fastq.gz \
  --outdir results/fastqc_trimmed --threads 4
```

Then re-run MultiQC, this time pointing it at the raw FastQC outputs, the fastp outputs, and the trimmed FastQC outputs. MultiQC will lay them out side-by-side so you can compare:

CLI

```
mkdir results/multiqc_post_trim
multiqc \
  results/fastqc/ results/fastp/ results/fastqc_trimmed \
  --outdir results/multiqc_post_trim/ \
  --filename multiqc_report_post_trim
```

Open `results/multiqc/post_trim/multiqc_report_post_trim.html` and inspect it.

4.5 Exercises

Exercise 4.1 — Default-parameter trimming

Run fastp on all samples using the loop above. For one sample of your choice, open the corresponding `<ACCESSION>_fastp.html` report and answer: a. What percentage of reads passed filtering? b. What percentage of reads had adapters detected and trimmed? c. What was the mean read length before vs after trimming? d. Was trimming aggressive, mild, or barely needed for this sample?

Exercise 4.2 — Stricter quality trimming

Pick a sample that has less than optimal quality metrics compared to the other samples. Re-run fastp on it with a stricter quality threshold (e.g., `--qualified_quality_phred 30` instead of 20), writing to a separate output directory so you don't overwrite the previous results:

```
mkdir results/fastp_strict
fastp --in1 ... --in2 ... --out1 results/fastp_strict/... \
      --qualified_quality_phred 30 --length_required 50 \
      --html results/fastp_strict/..._fastp.html \
      --json results/fastp_strict/..._fastp.json
```

Compare the strict report to the default-parameter report for the same sample: a. How much additional data was lost at Q30 vs Q20? b. Did the per-base quality profile of the surviving reads improve in a meaningful way? c. Would you use Q30 trimming for the whole dataset, or stick with Q20?

4.6 Check your understanding

? Check Your Understanding 4.1

a) fastp's overlap-based adapter detection finds adapters by looking for reverse-complementary overlap between R1 and R2. Why is this more sensitive than matching against a list of known adapter sequences?
b) If you set `--length_required 100` on 150-bp reads, what would you expect to happen?
c) Some pipelines trim much more aggressively than we did; for example, fixed 10-bp 5' clipping and Q30 quality trimming. Name one reason this might do more harm than good for the assembly workflow.

4.7 Further reading

Other available tools are:

- **Trimmomatic:** Still widely used in legacy pipelines. Slower, single-threaded by default, requires you to supply known adapter sequences.
- **Cutadapt:** the gold-standard adapter-trimmer for amplicon data, where adapter and primer sequences are known precisely.
- **BBDuk** (BBTools): supports kmer-based contaminant removal (e.g., for PhiX spike-in).
- **Trim Galore**: a wrapper around Cutadapt + FastQC for simple workflows.

There are various comparisons of these tools that can be found online. Take a moment to research these and look how they compare in terms of speed and accuracy for paired-end data.

5 Host read filtering with minimap2

Many bacterial sequencing workflows include a step to remove host reads, i.e. DNA from the human or animal source the bacterial isolate was collected from. For pure-culture Salmonella Typhi isolates the host contamination is usually <1% and might seem barely worth removing. We do the step anyway for two reasons: it is a useful introduction to read alignment and the SAM/BAM file format, and in clinical contexts (blood culture, tissue extracts) host removal is often essential for both privacy and ethics and downstream tool performance.

5.1 How host filtering works

The standard approach is to align all reads to a host reference genome, then keep only the reads that **fail to align**. Anything that mapped to the host is presumed to be host DNA and discarded; anything that did not map is presumed bacterial and retained. The risk is that a small fraction of true bacterial reads will be misaligned to the host genome by chance (false-positive host), at short read lengths this is rare but not zero.

We use **minimap2** for the alignment, because it is fast, accurate for short reads, and the same tool will work for long reads if you ever need to. We then use **samtools** to extract the unaligned read pairs.

5.2 A note on RAM and the host reference

The full human reference genome (GRCh38, ~3.1 Gb) builds a minimap2 index of approximately 12 GB, which may exceed the RAM capacity of standard laptops. Therefore, for this module, we will use a subset of the human ref genome index built from a subset of human chromosomes (`ref/GRCh38_subset.mmi`, ~2 GB). While with this approach some host reads may be missed (i.e., retained in the dataset), it should be negligible in this case. However, when running data originating from a real surveillance study, you should consider using the full reference. Host removal can generally be a complex task particularly for complex samples, so we recommend that you spend some time reading and understanding how it works, what tools are appropriate for your analysis (e.g., if you do metagenomics), etc.

5.3 Brief overview of SAM and BAM files

minimap2 produces alignments to the refernce genome in **SAM** format, which is a plain-text format with one line per alignment. If not reference genome is available, the data can also be stored unaligned. SAM files consist of an optional header section and an aligmnent section; the latter contains one record (single fragment alignment) per line describing the alignment between the fragment and the referecnce. Each record/line has a fixed set of 11 columns: query name, a bitwise flag describing alignment status, the reference sequence name, the reference position, the CIGAR string describing how the read aligned, and several others. More information can be found here: https://samtools.github.io/hts-specs/SAMv1.pdf. The compressed binary equivalent of SAM is **BAM**.

Each line in the header section starts with a @ followed by a two-letter record type code which is defined in the SAM format specificaion document. The alignment section contains one line per read alignment with multiple columens, with the first 11 being mandatory (QNAME, FLAG, RNAME, POS, MAPQ, CIGAR, MRNM, MPOS, ISIZE, SEQ, QUAL, OTHER).

In this case, the bitwise flag is the field we care about. Each bit encodes a property of the alignment: bit 4 (= 4) means *this read did not map*; bit 8 (= 8) means *the mate did not map*. The combination 4 + 8 = 12 means *neither read in this pair mapped to the reference*. For host filtering we want pairs where *both* reads are unmapped, so we filter on flag 12.

samtools provides flag-based filtering with `-f` (keep alignments where these flag bits are set) and `-F` (remove alignments where these flag bits are set). The canonical "both unmapped" filter is `-f 12 -F 256`, where `-F 256` additionally excludes secondary alignments to avoid double-counting.

5.4 Mapping and extracting reads

We can run this in a single command using the pipe symbol (|) to direct the output of one command to the next: minimap2 outputs SAM, samtools view filters and converts to BAM in memory, samtools fastq writes the unmapped pairs back out as gzipped FASTQ. No intermediate files needed.

CLI

```
minimap2 -ax sr -t 4 ref/GRCh38_subset.mmi \
    results/fastp/ERR4025225_1.trim.fastq.gz \
    results/fastp/ERR4025225_2.trim.fastq.gz \
  | samtools view -b -f 12 -F 256 - \
  | samtools fastq \
      -1 results/host_filtered/ERR4025225>_1.fastq.gz \
      -2 results/host_filtered/ERR4025225_2.fastq.gz
```

A breakdown of the options:

- `-ax sr` is the minimap2 preset for Illumina short reads (`a` requests SAM output, `x sr` selects short-read parameters).
- `-t 4` uses 4 threads.
- `samtools view -b -f 12 -F 256` keeps only paired-end records where both reads are unmapped, in BAM format.
- `samtools fastq -1 / -2` writes the two reads of each pair to separate files.

You should see progress messages from minimap2 ending with some metrics.

Exercise 5.1 — Host filter one sample

Pick a sample and run the host-filtering on it. Make a note of the following:

a. The total number of reads before filtering (in the trimmed FASTQ).
b. The total number of reads retained after host filtering.
c. The percentage of reads removed as host.
d. Is the host percentage what you would expect for a cultured isolate?

5.5 Looping over all samples

Let's have a look at all samples now. Again, a `for` loop can simplify and speed up things:

CLI

```
for r1 in results/fastp/*_1.trim.fastq.gz; do
  acc=$(basename "$r1" _1.trim.fastq.gz)
  minimap2 -ax sr -t 4 ref/GRCh38_subset.mmi
   ↪  results/fastp/${acc}_1.trim.fastq.gz
   ↪  results/fastp/${acc}_2.trim.fastq.gz \
    | samtools view -b -f 12 -F 256 - \
    | samtools fastq \
        -1 results/host_filtered/${acc}_1.fastq.gz \
        -2 results/host_filtered/${acc}_2.fastq.gz
done
```

This step will take a few minutes to complete and it depends on read count.

5.6 Counting how many reads were removed

After the loop finishes, you can count the reads in the input and output to see how many were removed per sample. For this, we would probably want a CSV spreadsheet that contains the sample name, the input and output reads, and the percentage of host reads found, i.e. a header that look something like this: `"sample,input_reads,kept_reads,host_pct"`. Then it would be useful to use another `for` loop that takes each sample/file name and counts lines in each (decompressed) file. For the latter, remember that each record in a fastq file is always 4 lines, so we need to divide the number of lines by 4. We do this for both the pre and post filtered files. Finally, it would probably be easier to interpret if we view the host read counts as a percentage. To get the `pct` value, we use `awk` to get the values we need, and then do some simple math to get the percentage of reads removed (i.e., classified as host). 'printf "%.2f"' rounds to two decimal points.

We have put this `for` loop together below and you can use it to get these numbers:

CLI

```
echo "sample,input_reads,kept_reads,host_pct"
for r1 in results/fastp/*_1.trim.fastq.gz; do
  acc=$(basename "$r1" _1.trim.fastq.gz)
  before=$(zcat "$r1" | wc -l)
  before=$((before / 4))
  after=$(zcat "results/host_filtered/${acc}_1.fastq.gz" | wc -l)
  after=$((after / 4))
  pct=$(awk "BEGIN { printf \"%.2f\", 100 - 100*$after/$before }")
  echo "${acc},${before},${after},${pct}"
done
```

Think how you can extract this table into a csv format, for example named host_filter_summary.csv. As a note, an alternative tool that could be used here is `seqkit stats`.

For pure bacterial cultures, we expect to see low host percentages. Anything above a certain threshold, say 5%, should be flagged for follow-up, as it may indicate co-extracted host DNA from a tissue sample, or a sample-tracking error.

Exercise 5.2 — Are any samples suspicious?

After running the loop over all samples and obtain read counts, identify any problematic, if any. What could be biological or technical explanation(s) for samples with high host read percentage?

5.7 Check your understanding

? Check Your Understanding 5.1

a) We filtered on samtools flag `-f 12`, meaning both reads in the pair are unmapped. Why not just use `-f 4` (this read unmapped), which is simpler?
b) Could host filtering accidentally remove genuine bacterial reads? Under what circumstances would this be more likely?
c) Why do we use the `-ax sr` preset for minimap2 rather than the default? What would the default preset be optimised for?

5.8 Further reading

Other tools performing these tasks are:

- **bowtie2**: the long-standing aligner for short reads. More precise than minimap2 (fewer false-positive alignments) but ~3× slower. The standard host-filter recipe in older pipelines.
- **Hostile**: a host-removal tool designed for clinical/public-health contexts. Uses a pre-built compact masked human reference (~5 GB) optimised to retain bacterial reads even at the cost of letting through a few host reads.
- **BWA-MEM2**: another short-read aligner, comparable to minimap2 for short reads. Slightly more memory intense.
- **kraken2 –paired –classified-out**: an entirely different approach using k-mer classification rather than alignment. Can be faster but requires a large database.

Again, it is worth having a look at publications discussing and comparing these tools in order to make and educated selection when choosing a tool for your analysis.

6 Taxonomic classification with BactInspector

We have a set of bacterial reads that we *think* are *Salmonella* Typhi. We have some good indications for this already, such as the genome length and GC percetage (Where did we get this information from?). Now, let's confirm this assumption.

Species-level taxonomic classification is a routine check in any bacterial NGS workflow, even if we have laboratory confirmation: it can catch sample mislabelling, mixed cultures, contamination with closely related species, and the occasional surprise of a completely different organism.

We will use **BactInspector**, a lightweight species caller from the Centre for Genomic Pathogen Surveillance (CGPS). It is much smaller and faster than the k-mer-based alternative Kraken2 that is commonly used, making it more feasible to run in the context of this workshop.

6.1 Why species confirmation matters

It is easy to assume that "I labelled this sample as Typhi when I sent it for sequencing, so the reads must be Typhi." In practice, this assumption fails in several scenarios:

- Sample-tracking errors during DNA extraction or library prep. Two samples processed at the same time may be swapped accidentally.
- Mixed cultures that look pure on an agar plate but are actually two species (or two strains).
- Cross-contamination from another sample on the same sequencing run, especially at low frequency.
- Misidentification at the clinical lab before the sample even reached you, e.g. a non-typhoidal *Salmonella* mistakenly recorded as Typhi.

A species level check on every sample is important to check against all of these scenarios. For Typhi specifically, a downstream tool like Pathogenwatch and GenoTyphi will give you serovar level confirmation and lineage assignment (covered later in this workshop); BactInspector gives you the broader species level sanity check first.

6.2 How BactInspector works

BactInspector is used to determine the most probable species based on sequence using refseq an Mash and/or to determine the closest reference in refseq. It compares the mash sketch of your reads

(or assembly) against a curated reference set of bacterial mash sketches. A mash sketch is a small fingerprint of a genome. Comparing two sketches gives you the mash distance, a number between 0 and 1 where 0 means identical and 1 means completely unrelated. The output is a list of the top hits ranked by mash distance, with the best hit reported as the assigned species.

6.3 Running BactInspector

BactInspector can take either fastq reads or a fasta assembly as input. We will run it on the host-filtered reads from the previous section. The `check_species` option is the one we need for this. Try running for one sample first:

CLI

```
mkdir results/bactinspector
bactinspector check_species \
  -fq results/host_filtered/ERR4025225_1.fastq.gz
```

The output is a TSV file:

CLI

```
cat results/bactinspector/species_investigation_*.tsv
```

Have a look at the output. Is the species *Salmonella enterica* Typhi?

6.4 Running BactInspector on all samples

CLI

```
bactinspector check_species \
  -i results/host_filtered \
  -fq "*_1.fastq.gz" \
  -o results/bactinspector
```

This will have produced one row per sample with the individual metrics.

A quick note: you may have noticed that we only ran this on r1 reads. BactInspector uses Mash sketches. Because sequencing produces 20-50× coverage of the genome, R1 alone already contains

every genomic k-mer many times over. Adding R2 would produce a near-identical sketch and roughly double the runtime for no diagnostic gain. The story is different for read-by-read classifiers like Kraken2, where both mates contribute independent evidence.

6.5 Exercises

Exercise 6.1 — Confirm species for one sample

After running BactInspector on all samples:

a. What is the top hit species for each sample?
b. How many of the 1000 sketch hashes are shared with the reference? Can you find information what values to expect?
c. Do you have any non-Typhi samples? What species was called instead? Is the mismatch a closely related *Salmonella* serovar, a different *Enterobacterales* genus, or something more distant?
d. Do you trust the species calls?

6.6 Check your understanding

? Check Your Understanding 6.1

a) Mash compares hash sketches rather than full k-mer profiles. What is the trade off of this? Why is the trade-off acceptable for species-level classification?
b) Could BactInspector miss low-level contamination — say, 2% of reads from a different species mixed in with 98% Typhi? Why or why not? Compare with how Kraken2 (which classifies every individual read) would behave on the same input.

6.7 Further reading

- **Kraken2** with **MiniKraken2-8GB**: the k-mer-based alternative, very commonly used. Kraken is a taxonomic sequence classifier that assigns taxonomic labels to DNA sequences. Kraken examines the k-mers within a query sequence and uses the information within those k-mers to query a database. That database maps k-mers to the lowest common ancestor (LCA) of all genomes known to contain a given k-mer. Construction of a Kraken 2 standard database requires approximately 100 GB of disk space. The default database size is 29 GB (as of Jan. 2018), and you will need slightly more than that in RAM if you want to build the default database. Due to these computing requirements, we did not use it in this workshop. Classifies every individual read against a database; catches low-level contamination that

mash-based methods may miss. To find more information on how to set it up later, visit https://github.com/DerrickWood/kraken2/blob/master/docs/MANUAL.markdown.

- **Speciator**: BactInspector's successor, used by Pathogenwatch. Same mash-sketch idea but with a tiered reference library that handles closely related species more carefully. Open source at https://github.com/pathogenwatch-oss/speciator.
- **GTDB-Tk**: it uses the Genome Taxonomy Database. It also requires a very large database.

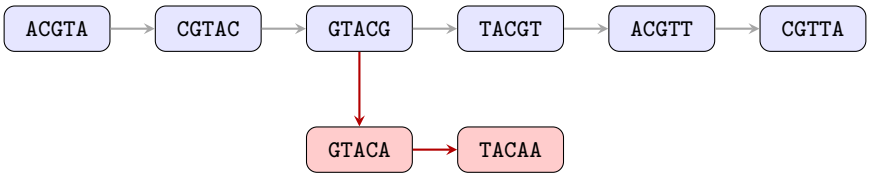
7 De novo assembly with shovill

The next step now is to reconstruct each genome from these short reads into an assembly. In comparison to alignment-based analyses, *de novo* assembly does not require a reference genome; it tries to figure out the genome sequence directly from the overlaps among the reads.

We will use **shovill**, a pipeline that uses SPAdes, an assembler that is tuned specifically for bacterial sequencing with Illumina paired-end reads. SPAdes is the assembler, while shovill handles read preparation, and parameter selection, and gives similar result in less time. Shovill can optionally trim reads, downsample reads to a target depth to increase speed without losing quality, chooses parameters for SPAdes and does a few post-assembly processes. Its defaults work well for most bacterial sequences without major adjustments, while SPAdes may occasionally offer finer grained control. Keep in mind that shovill is specifically for bacterial sequencing, and cannot be used, say, for metagenomes.

7.1 How short read *de novo* assembly works

Short read *de novo* assembly reconstructs genomes from reads without a reference by finding overlapping sequences to build longer **contigs**. The process uses **de Bruijn graphs**. The assembler breaks reads into **k-mers** (short overlapping sequences of fixed length k). Each unique k-mer becomes a node in the graph; two nodes are connected by an edge if they share k−1 bases and were observed consecutively in a read. In this way, the algorithm traverses the graph to find paths, merging overlapping reads into congiuous sequences called contigs and reconstructing the original sequence. In other words, overlapping k-mers are used like pieces of a puzzle to stitch together reads into (almost) complete genomes.



This schematic shows a simplified de Bruijn graph with each node being a 5-mer (k=5) and an edge showing that two k-mers sharing 4 bases adjacent in a read. If you follow the gray arrows through the graph, a contig is yield = ACGTACGTTA. However, if there is a branch (red arrows), which may be the cause of repeats or sequencing errors, break the path and end a contig.

In an ideal assembly, the graph would be one long long linear path per chromosome. However, n reality, it breaks into many short paths due to various reasons, such as:

- Repeats longer than the read length collapse identical k-mers from different genomic locations into the same node, creating branch points the assembler cannot resolve.
- Sequencing errors introduce spurious k-mers and false branches.
- Coverage gaps (regions with no reads) disconnect parts of the graph.

The output of a bacterial assembly is therefore usually not a single sequence but a set of contigs. The *quality* of an assembly is measured by how few and how long these contigs are.

7.2 Hardware constraints and parameter choices

SPAdes (and therefore shovill) is memory-intensive. The default `--ram 16 --cpus 4` setting will not fit on a 8 GB laptop which most of you may have. Therefore, we will use a more conservative configuration here:

- `--ram 8` caps memory at 8 GB (min required).
- `--cpus 2` uses 2 cores, but you can go also up to 4.
- `--depth 80` downsamples to 80× coverage (vs the default 150×). Bacterial isolate assemblies remain very good at 80× and the speedup is significant.
- `--gsize 4.8M` provides the expected genome size, skipping shovill's mash-based size estimation step.

With these parameters, each sample will take significant time on a typical laptop. Doing all samples in a single sitting would take a long time on its own, so we provide pre-reconstructed assemblies for the majority of the samples in `results/assemblies/`. You will assemble a couple of samples yourselves to feel the process; we will use the prepared assemblies for later steps.

7.3 Running shovill

CLI

```
shovill \
  --R1 results/host_filtered/ERR4025225_1.fastq.gz \
  --R2 results/host_filtered/ERR4025225_2.fastq.gz \
  --outdir results/assemblies/ERR4025225 \
  --depth 80 \
  --ram 6 \
  --cpus 2 \
  --gsize 4.8M
```

While it runs you will see a stream of progress messages and it will take a few minutes to run. Check which prepared assemblies you have and for which there are assemblies missing and run the same command for those samples, adjusting the input file names.

When shovill finishes, each output directory contains (where ACCESSION replaces the sample name below):

```
ls results/assemblies/<ACCESSION--depth>/
```

```
contigs.fa          # the final assembly in FASTA format
contigs.gfa         # the assembly graph (for visualization in Bandage)
shovill.corrections # which positions were polished
shovill.log         # full log of the run
spades.fasta        # raw SPAdes output before shovill polishing
```

`contigs.fa` is the file we care about as it is the final assembly. Take a look at the first few headers:

CLI

```
grep '>' results/assemblies/<ACCESSION>/contigs.fa | head -n 5
```

```
>contig00001 len=488942 cov=15.6 corr=0 origname=NODE_1_length_488942_cov_15.580507_pilon
>contig00002 len=419982 cov=16.6 corr=0 origname=NODE_2_length_419982_cov_16.616939_pilon
>contig00003 len=250349 cov=16.2 corr=0 origname=NODE_3_length_250349_cov_16.182011_pilon
>contig00004 len=249607 cov=16.4 corr=0 origname=NODE_4_length_249607_cov_16.408023_pilon
>contig00005 len=244922 cov=16.4 corr=0 origname=NODE_5_length_244922_cov_16.441557_pilon
```

Each header tells you the contig length, average coverage, and number of polishing corrections. Think about and discuss how many contigs you think will be a 'good' number, as well as of what length. Take into consideration the length of the genome. We will quantify this properly in the next chapter.

7.4 Exercises

Exercise 7.1 — Assemble one sample

a. How long did the run take?
b. How many contigs did it produce? (`grep -c '>' results/assemblies/<ACCESSION--depth>/contigs.fa`)
c. What is the total length of the assembly? (sum of contig lengths, e.g. `seqkit stats results/assemblies/<ACCESSION>/contigs.fa`)
d. What is the length of the largest contig?

Exercise 7.2 — Compare with the pre-computed assemblies

Inspect a few of the provided assemblies in `results/assemblies/`. a. What is the typical range for total assembly length? Does any sample look unusually short or unusually long? b. Do the contig count and longest-contig length vary much between samples? What could cause one isolate to assemble into ten contigs and another into a hundred?

7.5 Assembling all samples (optional)

If you have the time and patience you can assemble all samples in a loop after the workshop:

```
for r1 in results/host_filtered/*_1.fastq.gz; do
  acc=$(basename "$r1" _1.fastq.gz)
  shovill \
    --R1 results/host_filtered/${acc}_1.fastq.gz \
    --R2 results/host_filtered/${acc}_2.fastq.gz \
    --outdir results/assemblies/${acc} \
    --gsize 4.8M --depth 80 \
    --ram 8 --cpus 4
done
```

7.6 Check your understanding

? Check Your Understanding 7.1

a) Why does shovill downsample to a target depth rather than using all available reads? Surely more data is better?
b) A bacterial isolate assembly typically produces several dozen to several hundred contigs rather than a single chromosome. Name two distinct reasons the assembly fragments rather than running end-to-end.
c) When would you specifically **not** use shovill, and what should you use instead?

7.7 Further reading

gfa graphs can be visualised in a tool called Bandage, which will be demonstrated during the course of the workshop. Bandage is an interactive viewer for assembly graphs and it can be very useful for understanding why and how an assembly fragmented at specific places.

There are many other assembles that can be used for de novo assembly, also taking into consideration whether you have short reads only, long reads only, or both.

8 Assembly quality assessment with QUAST

A draft assembly is not automatically a *good* assembly. For example, an assembly: - can have the right total length but be split into thousands of tiny contigs - can have a few large contigs but contain misassemblies that link distant parts of the genome together incorrectly - can have unusual GC content suggesting contamination that earlier QC missed.

In this section, we will evaluate the assemblies using **QUAST**, a QC tool for assemblies, and aggregate the outputs into a final MultiQC report.

8.1 Reference-free vs reference-based metrics

QUAST reports two sets of metrics.

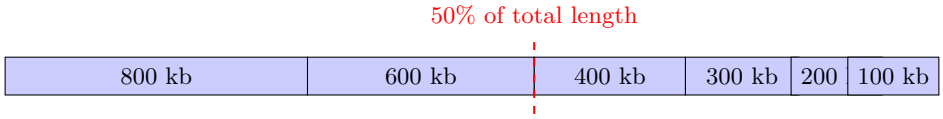
Reference-free metrics describe the assembly in isolation: total length, number of contigs, longest contig, GC%, and contiguity statistics like N50 and L50. These are always available and answer the question "is this assembly *internally* well-structured?"

Reference-based metrics require a reference genome and compare the assembly against it: misassembly count, mismatches per 100 kb, indels per 100 kb, genome fraction recovered, duplication ratio. These answer the question "is this assembly *correct* relative to a known good genome?" They are only meaningful if a close-enough reference exists. For *Salmonella* Typhi we have well characterised and annotated references such as the CT18 strain, which we will use for this purpose.

8.2 Understanding N50

N50 is considered an important reference-free statistic that defines the assembly quality in terms of contiguity. To get the N50, you sort the contigs from longest to shortest, then walk down the list summing lengths until you reach half the total assembly length. The length of the contig at which you cross that 50% mark is the N50. Larger N50 means a more contiguous assembly, in other words the genome is reconstructed in fewer, longer fragments.

L50 is a related metric: how many contigs you need (starting from the longest) to cover half the assembly. Generally, lower L50 is better. For example, L50 = 2 means that just 2 contigs cover half the genome.



Total = 2.4 Mb. Half-total = 1.2 Mb is reached partway through the 600-kb contig.

N50 = 600 kb the length of the contig containing the half-total point.

L50 = 2 the number of contigs needed to reach half the total.

In other words, once the contigs are arranged from longest to shortest, N50 is the contig length at which cthe umulative coverage crosses 50% of the total assembly length, while L50 is the count of contigs up to and including that one.

Other output metrics of QUAST are more self-explanatory, like the total assembly length, largest contg length, GC percentage, mismatch rates (when a ref is provided), etc. Full descriptions can be found in the manual https://quast.sourceforge.net/docs/manual.html#sec3.1.

8.3 Running QUAST without a reference

QUAST can take many assemblies as input and produce a comparative report covering all of them. The simplest run is reference-free:

CLI

```
quast.py results/assemblies/*/contigs.fa \
  --output-dir results/quast/no_ref \
  --threads 4 \
  --labels $(ls results/assemblies/ | tr '\n' ',' | sed 's/,$//')
```

The `--labels` argument gives each assembly a clean sample name in the report (otherwise QUAST uses the path `results_assemblies_X_contigs`). Have a look at the output directory:

CLI

```
ls results/quast/no_ref/
```

```
report.html          # interactive HTML report
report.pdf           # printable PDF report
report.tsv           # tab-separated values
report.txt           # plain-text report
```

```
report.tex           # LaTeX version of report
icarus.html          # contig browser (one of QUAST's nicer features)
quast.log
transposed_report.tsv # tranposed version of the report
```

Open `report.html` in your browser. The top of the report is a table where each column is a sample and each row is a metric:

Have a look at the report. Are assemblies comparable? Are there any outliers? How do the numbers of contigs look? The N50s?

8.4 Running QUAST with a reference

To get reference-based metrics, point QUAST at the CT18 *Salmonella* Typhi reference:

CLI

```
quast.py results/assemblies/*/contigs.fa \
  --output-dir results/quast/vs_CT18 \
  --reference ref/CT18_reference.fa \
  --threads 4 \
  --labels $(ls results/assemblies/ | tr '\n' ',' | sed 's/,$//')
```

The report now includes additional rows:

- **# misassemblies**: places where the assembly says a region from position X is adjacent to a region from position Y on the reference, when X and Y are actually far apart. A handful per assembly is normal; dozens indicate a structural problem.
- **# mismatches / 100 kbp**: base-level differences from the reference, excluding misassemblies. A few per 100 kbp reflects genuine variation between strains.
- **Genome fraction (%)**: proportion of the CT18 reference recovered by the assembly.
- **Duplication ratio**: total assembly length divided by reference length. Should be close to 1.0; significantly above suggests contamination or assembly errors creating duplicated content.

You will also notice additional sub-directories in the output directory. Have a look through these and look them up in the manual if something does not make sense.

8.5 Confirmatory BactInspector run on the assembly

A nice sanity check is to run BactInspector again, this time on the assembled contigs rather than the reads. The mash sketch of an assembly is cleaner than the sketch of reads (because of better k-mer coverage from contigs), so the species call is often sharper.

CLI

```
for d in results/assemblies/*/; do
  acc=$(basename "$d")
  bactinspector check_species \
    -fa "${d}contigs.fa" \
    -o results/bactinspector/${acc}_asm
done
```

Compare the top hits to the read-based BactInspector calls previously run. Do they agree? If not, further investigation is warranted.

8.6 The final MultiQC report

Run MultiQC one last time, pointing it at the full pipeline we ran It will pick up FastQC (raw and trimmed), fastp, QUAST, and samtools outputs if any were saved/retained:

CLI

```
multiqc \
  results/fastqc/ \
  results/fastp/ \
  results/fastqc/trimmed/ \
  results/quast/vs_CT18/ \
  --outdir results/multiqc/final \
  --filename final_report
```

Open `results/multiqc/final/final_report.html`. This HTML page shows how each of the samples behaved at every step of the pipeline. Spend a few minutes scrolling through. Each plot can be downloaded as PNG or SVG for reports.

8.7 Exercises

Exercise 8.1 — Rank the assemblies by quality

Run QUAST reference-free on all assemblies and open the HTML report. Based on N50, total length, and number of contigs (above some sensible threshold like 1 kb), produce a ranking:

a. Which sample has the best assembly? What metrics did you focus on to make your decision?
b. Which has the worst?
c. Are there any samples whose total length is suspiciously short (<4.5 Mb) or suspiciously long (>5.0 Mb)? What might cause each direction of deviation?

Exercise 8.2 — Compare best and worst against the reference

Run QUAST with the CT18 reference. Compare: a. How does the misassembly count differ between the best and worst assemblies? b. What is the genome fraction recovered for each? c. For the worst sample, look at the misassembly types in `report.txt` (or the Icarus viewer). Are they translocations, relocations, or inversions? What might that say about the underlying problem? d. Have you identified any issues previously (such as any unusual host content, outlier species, etc.)?

8.8 Check your understanding

? Check Your Understanding 8.1

a) Two assemblies have the same total length (4.78 Mb) but very different N50s (300 kb vs 30 kb). Which assembly is more useful for downstream analysis, and why?
b) A QUAST report shows 99.7% genome fraction recovered but 27 misassemblies. Should you trust this assembly? Explain.
c) Why did we re-run BactInspector on the assembly in addition to having run it on the reads? Did we expect the result to be different?

8.9 Further reading

- **BUSCO**: measures gene-content completeness by searching the assembly for a curated set of single-copy genes expected in a given taxonomic group. For *Salmonella* the relevant lineage is `enterobacterales_odb10` (~440 genes). BUSCO completeness >98% indicates a complete gene set; lower values suggest missing genomic regions even if the assembly looks contiguous.

- **Bandage**: interactive viewer for the `.gfa` assembly graphs. Lets you explore the assembly (and any fragments) in detail
- **Pathogenwatch**: — a Typhi-aware downstream web service that takes an assembly and reports the GenoTyphi lineage, AMR genes, and contextual placement against thousands of public genomes. We will discuss this later in the workshop.

9 Summary

In this module, we covered bacterial isolate QC and assembly tools, ran on real public-health-relevant data. The final MultiQC report you produced in the previous section should have now helped you to make decisions about which samples are good enough to take forward.

9.1 What we ran in this module

For each of the Typhi samples, you should now have:

- Raw read quality characterisation (FastQC).
- Adapter- and quality-trimmed reads (fastp) and a post-trim FastQC re-run for comparison.
- Host-filtered reads with a known percentage of host removed (minimap2).
- A confirmed species call (BactInspector).
- A draft assembly of approximately 4.8 Mb (shovill).
- Assembly quality metrics relative to the *Salmonella* Typhi CT18 reference (QUAST).
- A confirmatory species call on the assembly itself.

This set of data can serve as evidence that the samples are (or not) what we think them to be and the sequencing data are of good quality for downstream typing and analysis

9.2 Some points to think about during future implementation

There are no right answers; the goal is to think about the assumptions and judgement calls you will need to be making.

1. **Outliers.** Across these types of analyses, you may flag samples for various reasons, e.g. bad per-base quality, high host percentage, unexpected species call, fragmented assembly, etc. Did the same samples keep showing up as problematic at multiple steps? If so, what does that pattern say?
2. **Defaults vs. tuning.** We mostly used default or near-default parameters for every tool. Where, specifically, do you think the workshop's defaults are too lax for other types of usage? Where are they too strict? When would you consider adjusting the thresholds and other parameters?

3. **What you can do next.** Which sample(s) do you think are of sufficient quality to proceed to next steps, and which would you reject? Why? Do you need to repeat sequencing in these cases? How can you improve assembly efficiency?
4. **Computational constraints.** If you had access to a powerful server, with enough memory and RAM, which of analyses would you reconsider or do differently?

9.3 Short summary quiz

?

Try these without referring back to the chapters.
Q1. Can you identify plasmids from an assembly graph?
Q2. What does a Phred Q-score of 30 mean in terms of basecalling accuracy?
Q3. What kind of information do you get from a fastq, a fasta, a sam, and a bam file?
Q4. Two assemblies have N50s of 250 kb and 25 kb but the same total length of 4.78 Mb. Which is more useful for downstream analysis, and what specifically can the higher-N50 assembly do that the lower-N50 one may struggle with?
Q5. Name two reasons a bacterial assembly fragments into many contigs rather than as a single chromosome.

9.4 What can you do next

QC and assembly are the foundation of bacterial whole-genome surveillance, but will probably not give you enough information. Next, you can run quite a few types of analyses:

- **Reference-based variant calling** for SNP-based outbreak investigation.
- **Annotation** to identify the presence of genes, regulatory features, and mobile genetic elements.
- **AMR gene detection** against curated antimicrobial resistance databases.
- **Plasmid detection** against plasmid databases.
- **Typing** and **Lineage assignment** by MLST, cgMLST, assigning genotyped clades.
- **Phylogenetic context** by placing your isolates within a public (and global) collection of many Typhi genomes and interpreting against available metadata.

Subsequent modules in this workshop series cover many of these steps. The QC and assembly habits you have developed today — sceptical inspection at every step, side-by-side comparison via MultiQC, multiple cross-checks on species identity — apply to all of them.

9.5 A few last practical points:

- **Keep a good record of QC reports and which samples you excluded from further analysis and why.** It is important to know whether you excluded samples and why; this information is useful for collaborators and essential to research integrity.
- **The `results/` directory is large** (a few GB) but most of it is intermediate output. The files you will actually need to keep long-term are the trimmed FASTQ pairs, the assemblies, and the final MultiQC report. Everything else can be quickly regenerated.
- **Record the tool versions you have used.** It is very good practice to record tool versions every time you run them. This is essential for reproducibility and reporting in publications.

Thank you for working through the module. Feedback — what worked, what was confusing, what should be added or removed — is gratefully received by the workshop organisers.